

Algoritmos e Programação de Computadores I

Comandos de Repetição em C

Prof. Marcelo Farias

Objetivos da Aula

- Compreender as estruturas de repetição em C.
- Conhecer a sintaxe e o funcionamento de cada comando de repetição.
- Aprender a utilizar e diferenciar laços de repetição determinados e indeterminados.

Comandos de Repetição

- Permitem que um comando ou bloco de comandos seja executado enquanto uma condição for verdadeira.
- Também são chamados de laços.
- C suporta os comandos for, while e do-while.

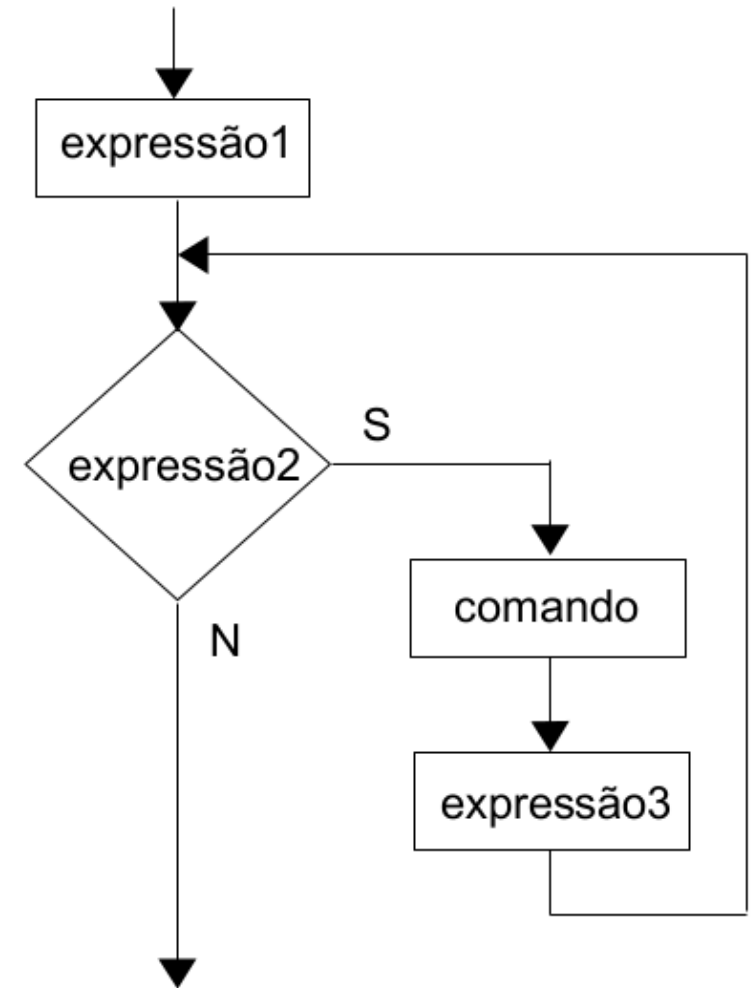
Comando for

- Comando de repetição determinada.
- Um contador controla a quantidade de vezes que um comando ou bloco de comandos é executado.
- A variável do contador é incrementada ou decrementada até chegar na condição de parada.

Sintaxe do for

for(expressão1; expressão2; expressão3) comando;

```
for(expressão1; expressão2; expressão3)  
{  
    comando1;  
    comando2;  
    ...  
    comandoN;  
}
```



Exemplo de for

```
for(int i = 0; i < 10; i++) printf("%d\n", i); // 0 a 9 com salto de 1
```

```
for(int i = 10; i > 0; i--) printf("%d\n", i); // 10 a 1 com salto de -1
```

```
for(int i = 0; i < 10; i += 2) { // salto de 2  
    printf("%d\n", i);  
}
```

```
for(int i = 10; i > 0; i -= 3) { // salto de -3  
    printf("%d\n", i);  
}
```

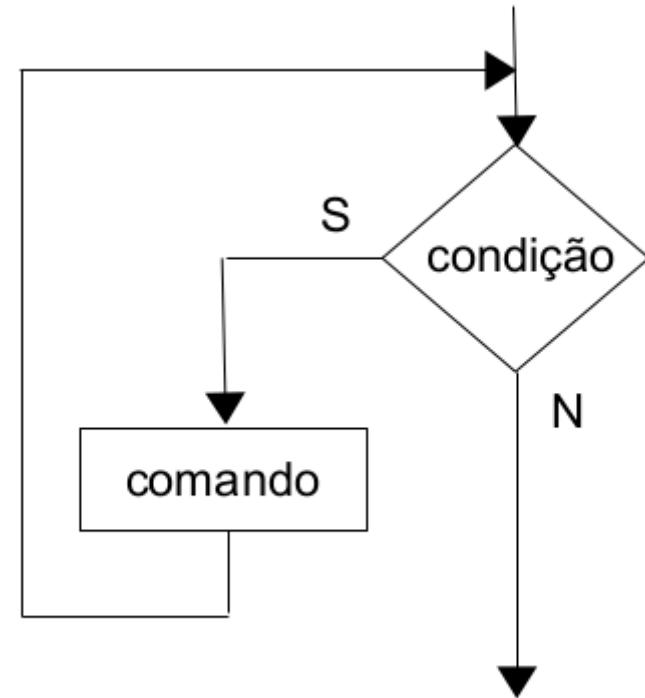
Comando while

- Comando de repetição indeterminada.
- Enquanto uma condição for verdadeira, um comando ou bloco de comandos será executado.
- Não requer um contador.

Sintaxe do while

`while(condição) comando;`

```
while(condição) {  
    comando1;  
    comando2;  
    ...  
    comandoN;  
}
```



Exemplo de while

```
int i = 0;
while(i < 10) printf("%d\n", i++);

char ch = getchar(); // ler uma tecla
while(ch != '\n') {
    printf("%c\n", ch);
    ch = getchar();
}
```

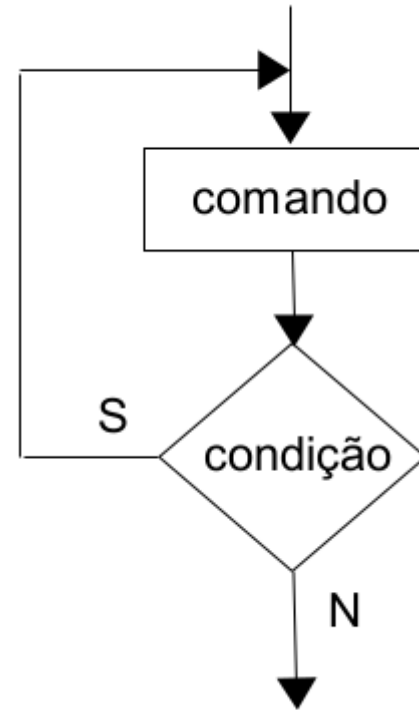
Comando do-while

- Diferencia-se do comando while porque a condição é verificada após a execução do bloco.
- É executado pelo menos uma vez, independentemente da condição inicial.

Sintaxe do do-while

do *comando*; while(*condição*);

```
do {  
    comando1;  
    comando2;  
    ...  
    comandoN;  
} while(condição);
```



Exemplo de do-while

```
int i = 0;  
do printf("%d\n", i++); while(i < 10);
```

```
char ch = '\0';  
do {  
    ch = getchar(); // ler uma tecla  
    printf("%c\n", ch);  
} while(ch != '\n');
```

Laços Aninhados

- Um comando de repetição declarado dentro de outra repetição.
- Tipicamente usados para percorrer elementos de matrizes.
- A quantidade de laços aninhados afeta o desempenho do programa.

Exemplo de Laço Aninhado

```
for(int i = 0; i < 10; i++)  
    for(int j = 0; j < 10; j++)  
        printf("%d, %d\n", i, j);  
  
int i = 0;  
while(i < 10) {  
    int j = 0;  
    do {  
        printf("%d, %d\n", i, j);  
        j++;  
    } while(j < 10);  
    i++;  
}
```

Laço Infinito

- Laço é repetido infinitamente, ou seja, sem parar.
- Uma condição de parada deve ser definida no bloco de comandos do laço.

Exemplo de Laço Infinito

```
for(;;) {  
    printf("Ao infinito e além!\n");  
}
```

```
while(1) {  
    printf("Ao infinito e além!\n");  
}
```

```
do {  
    printf("Ao infinito e além!\n");  
} while(1);
```


Comando break

- Interrompe a repetição do laço.
- Essencial para controlar a execução de laços infinitos.

Exemplo de break

```
int i = 0;
for(;;) {
    i++;
    if(i==10) break; // condição de parada
}
```

```
char ch = '\0';
while(1) {
    ch = getchar();
    if(ch=='\n') break; // condição de parada
}
```

Comando continue

- Salta uma repetição do laço.
- Retorna para início do laço não executando os comandos abaixo dele.

Exemplo de continue

```
for(int i=0; i < 10; i++) {  
    if(i > 5) continue; // salta a repetição no laço 5  
    printf("%d, ", i);  
}
```

```
char ch = '\0';  
do {  
    ch = getchar(); // ler uma tecla  
    if(ch < '0' || ch > '9') continue; // salta a repetição se não for dígito  
    printf("%c\n", ch);  
} while(ch != '\n');
```

Em Resumo

- O **for** é ideal para repetições com número conhecido ou controlado por contador.
- O **while** repete enquanto a condição for verdadeira, podendo não executar nenhuma vez se inicialmente falsa.
- O **do-while** assegura pelo menos uma execução, pois verifica a condição após o bloco.

Mapa da aula

Aula (teoria): Aula11–12 — Repetição

Laboratório guiado: Pratica07

Atividade avaliativa: Atividade03 (opções 2 e 3)

Sugestão de ritmo:

1) Conceitos nos slides 2) Prática no lab 3) Atividade sem 'receita'

Para estudar no livro

Livro-base: ANTONELLO, Ricardo. Linguagem C. 1. ed.

Leitura indicada: Cap. 6 — Laços de repetição (p. 68–88)

Exercícios do livro: Exercícios 6.5

Revise o capítulo após a aula e antes da prática.

Desafio (8 min)

Leia números até 0 e mostre a quantidade de pares. Escolha while ou do-while e justifique.

Trabalhe em dupla (instrução por pares).

Ao final, uma dupla compartilha a solução com a turma.

Quiz rápido (3 perguntas)

1) Diferença entre while e do-while?

2) Para que serve continue?

3) Como interromper um for(;;) ?

Respostas no quadro / Mentimeter / Kahoot.

Referências

- SCHILDT, Herbert. C: Completo e Total. 3. ed. Pearson Makron Books, 1997.
- ANTONELLO, Ricardo. Linguagem C. 1. ed. Livro Digital, 2020.